

Reinforcement Learning: Setting Up Gymnasium Environments

Lab 1 Task Report

Submitted by:

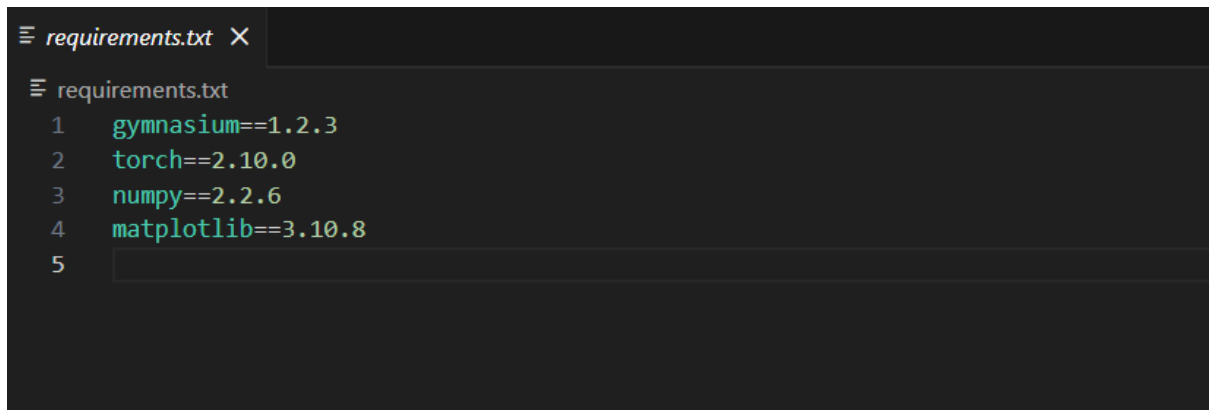
Junaid Asif

Roll No: BSAI-144

Instructor:

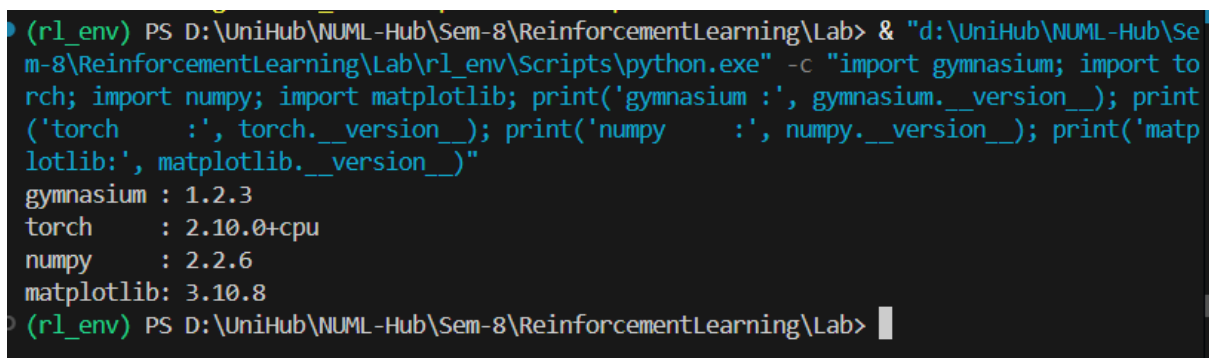
Ms. Hina Ali

February 20, 2026

A screenshot of a code editor window titled 'requirements.txt'. The editor shows a list of pinned library versions: gymnasium==1.2.3, torch==2.10.0, numpy==2.2.6, and matplotlib==3.10.8. The line numbers 1 through 5 are visible on the left side of the editor.

```
requirements.txt
1 gymnasium==1.2.3
2 torch==2.10.0
3 numpy==2.2.6
4 matplotlib==3.10.8
5
```

Figure 1: requirements.txt with pinned library versions

A screenshot of a terminal window showing the execution of a Python script to verify installed packages. The script imports gymnasium, torch, numpy, and matplotlib, and prints their versions. The output shows the installed versions: gymnasium: 1.2.3, torch: 2.10.0+cpu, numpy: 2.2.6, and matplotlib: 3.10.8.

```
(rl_env) PS D:\UniHub\NUML-Hub\Sem-8\ReinforcementLearning\Lab> & "d:\UniHub\NUML-Hub\Sem-8\ReinforcementLearning\Lab\rl_env\Scripts\python.exe" -c "import gymnasium; import torch; import numpy; import matplotlib; print('gymnasium :', gymnasium.__version__); print('torch :', torch.__version__); print('numpy :', numpy.__version__); print('matplotlib:', matplotlib.__version__)"
gymnasium : 1.2.3
torch      : 2.10.0+cpu
numpy      : 2.2.6
matplotlib: 3.10.8
(rl_env) PS D:\UniHub\NUML-Hub\Sem-8\ReinforcementLearning\Lab>
```

Figure 2: Installed packages verification (pip list)

```

env1_cartpole.py X
env1_cartpole.py > ...
4 # Environment 1: CartPole-v1
5 # -----
6
7 # Step 1: Create the environment (render_mode="human" opens a visual window)
8 env = gym.make("CartPole-v1", render_mode="human")
9
10 # Step 2: Print basic environment info
11 print("Environment Name :", "CartPole-v1")
12 print("Observation Space :", env.observation_space) # what the agent sees
13 print("Action Space      :", env.action_space)      # 0 = push left, 1 = push right
14
15 # Step 3: Reset the environment to get starting state
16 observation, info = env.reset()
17
18 print("\nStarting Observation:", observation)
19 print("  Cart Position :", observation[0])
20 print("  Cart Velocity :", observation[1])
21 print("  Pole Angle      :", observation[2])
22 print("  Pole Velocity :", observation[3])
23 print(f"Info          :", info)
24
25 # Step 4: Run 5 steps with random actions
26 print("\n--- Running 50 Steps ---")
27
28 for step in range(50):
29
30     # Pick a random action
31     action = env.action_space.sample()
32
33     # Take the action
34     observation, reward, terminated, truncated, info = env.step(action)
35
36     # Is the episode over?
37     done = terminated or truncated
38
39     # Print all variables
40     print(f"\nStep      : {step + 1}")
41     print(f"Action       : {action} (0=Left, 1=Right)")
42     print(f"Obs[0] Cart Position : {observation[0]:.4f}")
43     print(f"Obs[1] Cart Velocity : {observation[1]:.4f}")
44     print(f"Obs[2] Pole Angle    : {observation[2]:.4f}")
45     print(f"Obs[3] Pole Velocity : {observation[3]:.4f}")
46     print(f"Reward        : {reward}")
47     print(f"Terminated: {terminated}")
48     print(f"Truncated   : {truncated}")
49     print(f"Done        : {done}")
50     print(f"Info        : {info}")
51
52     if done:
53         observation, info = env.reset()
54
55 # Step 5: Close the environment
56 env.close()
57 print("\nDone!")
58

```

Figure 3: CartPole-v1 – Python code

```

(r1_env) PS D:\UniHub\NML-Hub\Sem-8\ReinforcementLearning\Lab> & D:\UniHub\NML-Hub\Sem-8\ReinforcementLearning\Lab\r1_env\Scripts\python.exe d:/UniHub/NML-Hub/Sem-8/ReinforcementLearning/Lab/env1_cartpole.py
Environment Name : CartPole-v1
Observation Space : Box([-4.8 -inf -0.41887903 -inf], [4.8 inf 0.41887903 inf], (4,), float32)
Action Space      : Discrete(2)

Starting Observation: [ 0.00113329  0.0417241 -0.01501006 -0.02944009]
  Cart Position : 0.0011332886
  Cart Velocity : 0.041724104
  Pole Angle    : -0.015010055
  Pole Velocity : -0.029440088
Info           : {}

--- Running 50 Steps ---

Step      : 1
Action     : 1 (0=Left, 1=Right)
Obs[0] Cart Position : 0.0020
Obs[1] Cart Velocity : 0.2371
Obs[2] Pole Angle    : -0.0156
Obs[3] Pole Velocity : -0.3268
Reward     : 1.0
Terminated: False
Truncated  : False
Done       : False
Info       : {}

```

Figure 4: CartPole-v1 – Terminal output showing all variable values

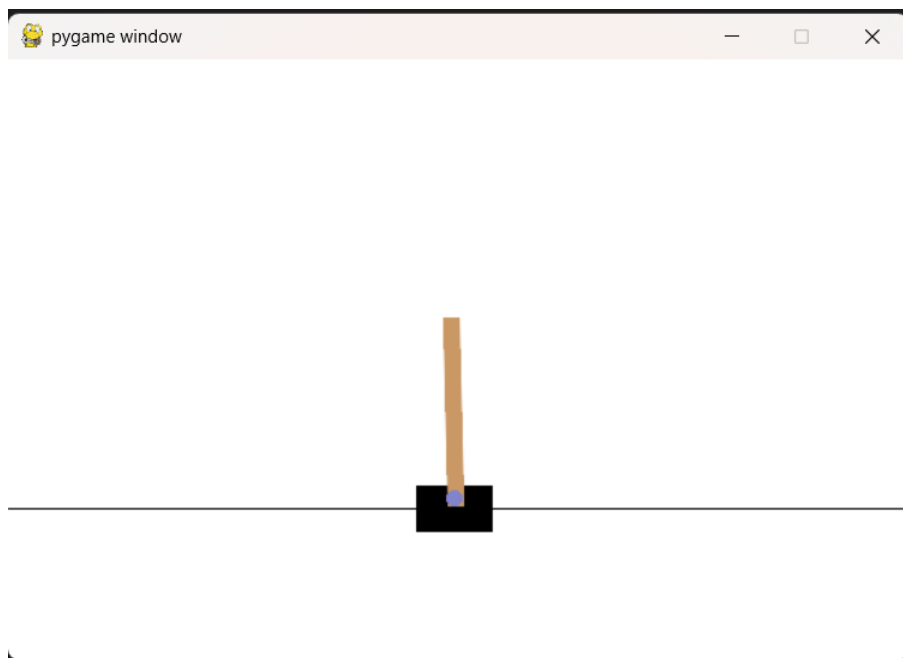


Figure 5: CartPole-v1 – Visual render window

```

env2_frozenlake.py > ...
1  import gymnasium as gym
2
3  # -----
4  # Environment 2: FrozenLake-v1
5  # -----
6
7  # Step 1: Create the environment (render_mode="human" opens a visual window)
8  env = gym.make("FrozenLake-v1", is_slippery=False, render_mode="human")
9
10 # Step 2: Print basic environment info
11 print("Environment Name :", "FrozenLake-v1")
12 print("Observation Space :", env.observation_space) # 16 tiles (4x4 grid)
13 print("Action Space      :", env.action_space)      # 0=Left 1=Down 2=Right 3=Up
14 print("Number of States  :", env.observation_space.n)
15 print("Number of Actions :", env.action_space.n)
16
17 # Step 3: Reset the environment
18 observation, info = env.reset()
19
20 print("\nStarting Observation (tile number):", observation)
21 print("Info :", info)
22 print(" prob = probability of this transition:", info.get("prob", "N/A"))
23
24 # Step 4: Run 5 steps with random actions
25 print("\n--- Running 50 Steps ---")
26
27 action_names = {0: "LEFT", 1: "DOWN", 2: "RIGHT", 3: "UP"}
28
29 for step in range(50):
30
31     # Pick a random action
32     action = env.action_space.sample()
33
34     # Take the action
35     observation, reward, terminated, truncated, info = env.step(action)
36
37     # Is the episode over?
38     done = terminated or truncated
39
40     # Print all variables
41     print(f"\nStep      : {step + 1}")
42     print(f"Action       : {action} ({action_names[action]})")
43     print(f"Observation  : {observation} (current tile on grid)")
44     print(f"Reward       : {reward} (1 = reached goal, 0 = otherwise)")
45     print(f"Terminated   : {terminated}")
46     print(f"Truncated    : {truncated}")
47     print(f"Done         : {done}")
48     print(f"Info         : {info}")
49
50     if done:
51         print("Episode ended. Resetting...")
52         observation, info = env.reset()
53
54 # Step 5: Close the environment
55 env.close()
56 print("\nDone!")
57

```

Figure 6: FrozenLake-v1 – Python code

```

(r1_env) PS D:\UniHub\NUML-Hub\Sem-8\ReinforcementLearning\Lab> & D:/UniHub/NUML-Hub/Sem-8/ReinforcementLearning/Lab/r1_env/Scripts/python.exe d:/UniHub/NUML-Hub/Sem-8/ReinforcementLearning/Lab/env2_frozenlake.py
Environment Name : FrozenLake-v1
Observation Space : Discrete(16)
Action Space : Discrete(4)
Number of States : 16
Number of Actions : 4

Starting Observation (tile number): 0
Info : {'prob': 1}
prob = probability of this transition: 1

--- Running 50 Steps ---

Step : 1
Action : 1 (DOWN)
Observation : 4 (current tile on grid)
Reward : 0 (1 = reached goal, 0 = otherwise)
Terminated : False
Truncated : False
Done : False
Info : {'prob': 1.0}

Step : 2
Action : 0 (LEFT)
Observation : 4 (current tile on grid)
Reward : 0 (1 = reached goal, 0 = otherwise)
Terminated : False
Truncated : False
Done : False
Info : {'prob': 1.0}

Step : 3
Action : 0 (LEFT)
Observation : 4 (current tile on grid)
Reward : 0 (1 = reached goal, 0 = otherwise)
Terminated : False
Truncated : False
Done : False
Info : {'prob': 1.0}

```

Figure 7: FrozenLake-v1 – Terminal output showing all variable values



Figure 8: FrozenLake-v1 – Visual render showing the 4x4 grid

```

env3_mountaincar.py > ...
1  import gymnasium as gym
2
3  # -----
4  # Environment 3: MountainCar-v0
5  # -----
6
7  # Step 1: Create the environment (render_mode="human" opens a visual window)
8  env = gym.make("MountainCar-v0", render_mode="human")
9
10 # Step 2: Print basic environment info
11 print("Environment Name :", "MountainCar-v0")
12 print("Observation Space :", env.observation_space) # position and velocity
13 print("Action Space      :", env.action_space)      # 0=Left 1=Nothing 2=Right
14 print("Position Range   : min =", env.observation_space.low[0], "max =", env.observation_space.high[0])
15 print("Velocity Range   : min =", env.observation_space.low[1], "max =", env.observation_space.high[1])
16
17 # Step 3: Reset the environment
18 observation, info = env.reset()
19 |
20 print("\nStarting Observation:", observation)
21 print(" Position :", observation[0])
22 print(" Velocity :", observation[1])
23 print("Info      :", info)
24
25 # Step 4: Run 5 steps with random actions
26 print("\n--- Running 50 Steps ---")
27
28 action_names = {0: "PUSH LEFT", 1: "NO PUSH", 2: "PUSH RIGHT"}
29
30 for step in range(50):
31
32     # Pick a random action
33     action = env.action_space.sample()
34
35     # Take the action
36     observation, reward, terminated, truncated, info = env.step(action)
37
38     # Is the episode over?
39     done = terminated or truncated
40
41     # Print all variables
42     print(f"\nStep      : {step + 1}")
43     print(f"Action       : {action} ({action_names[action]})")
44     print(f"Position      : {observation[0]:.4f} (car position on the hill)")
45     print(f"Velocity      : {observation[1]:.4f} (car speed)")
46     print(f"Reward        : {reward} (-1 every step until goal reached)")
47     print(f"Terminated     : {terminated}")
48     print(f"Truncated      : {truncated}")
49     print(f"Done           : {done}")
50     print(f"Info           : {info}")
51
52     if done:
53         print("Episode ended. Resetting...")
54         observation, info = env.reset()
55
56 # Step 5: Close the environment
57 env.close()
58 print("\nDone!")
59

```

Figure 9: MountainCar-v0 – Python code

```

(r1_env) PS D:\UniHub\NMPL-Hub\Sem-8\ReinforcementLearning\Lab> & D:/UniHub/NMPL-Hub/Sem-8/ReinforcementLearning/Lab/r1_env/Scripts/python.exe d:/UniHub/NMPL-Hub/Sem-8/ReinforcementLearning/Lab/env3_mountaincar.py
Environment Name : MountainCar-v0
Observation Space : Box([-1.2 -0.07], [0.6 0.07], (2,), float32)
Action Space : Discrete(3)
Position Range : min = -1.2 max = 0.6
Velocity Range : min = -0.07 max = 0.07

Starting Observation: [-0.47174078  0.          ]
Position : -0.47174078
Velocity : 0.0
Info : {}

--- Running 50 Steps ---

Step : 1
Action : 1 (NO PUSH)
Position : -0.4721 (car position on the hill)
Velocity : -0.0004 (car speed)
Reward : -1.0 (-1 every step until goal reached)
Terminated : False
Truncated : False
Done : False
Info : {}

Step : 2
Action : 2 (PUSH RIGHT)
Position : -0.4719 (car position on the hill)
Velocity : 0.0002 (car speed)
Reward : -1.0 (-1 every step until goal reached)
Terminated : False
Truncated : False
Done : False
Info : {}

Step : 3
Action : 2 (PUSH RIGHT)
Position : -0.4711 (car position on the hill)
Velocity : 0.0008 (car speed)
Reward : -1.0 (-1 every step until goal reached)
Terminated : False
Truncated : False
Done : False
Info : {}

```

Figure 10: MountainCar-v0 – Terminal output showing all variable values

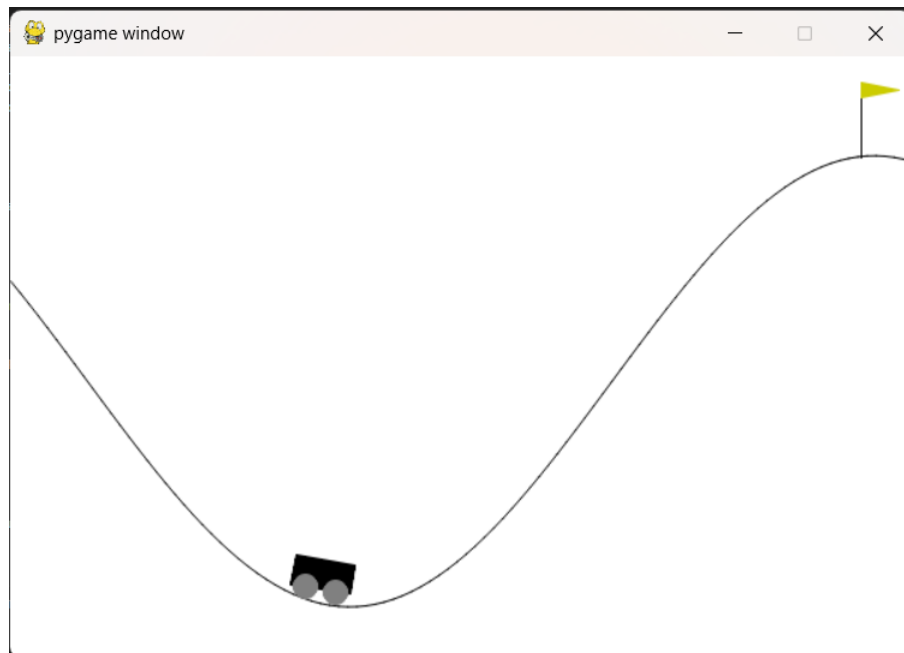


Figure 11: MountainCar-v0 – Visual render showing the car on the hill